

Next.js

What is next js..?

“Next.js is a React framework that enables server-side rendering, static site generation, and full-stack capabilities. It builds on React by providing routing, performance optimizations, and API routes out-of-the-box, making it ideal for SEO-friendly and high-performance web applications.”

Why Next over React?

React alone handles only the UI, so it struggles with SEO, slow first-page load, and routing setup. Next.js solves these by providing server-side rendering (SSR) for SEO, static site generation (SSG) for performance, and built-in file-based routing and API routes—giving a full-stack-ready, high-performance app out-of-the-box.”

1 CSR — Client-Side Rendering

- How it works:
 1. Browser requests the page.
 2. Server sends a bare-bones HTML with JS files.
 3. JS runs in the browser and builds the entire UI dynamically.

Key point: Everything renders after JS executes on the client.

Pros:

- Fast navigation after initial load (single-page app behavior).
- Great for highly interactive UIs.

Cons:

- Slow first-page load.
- Poor SEO unless extra tools are used.
- Relies heavily on client’s device performance.

2 SSR — Server-Side Rendering

- How it works:
 1. Browser requests the page.
 2. Server runs React code and renders the full HTML on-the-fly.
 3. Server sends ready-to-display HTML + JS to hydrate for interactivity.
- Key point: Server does initial rendering, browser just hydrates.

Pros:

- Fast first-page load, good for SEO.
- Works well for dynamic content that changes per request.

Cons:

- Higher server load per request.
- Slightly slower than static after initial load.
- More complex caching required for performance.

3 SSG — Static Site Generation

- How it works:
 1. During build, Next.js runs your React code for each page.
 2. Generates static HTML files.
 3. Browser requests page → server sends pre-built HTML immediately.
- Key point: Pages are pre-rendered at build, extremely fast and SEO-friendly.

Pros:

- Extremely fast page loads, excellent SEO.
- Low server load (can even be served via CDN).

Cons:

- Not suitable for frequently changing content.
- Requires rebuilds to update content.

ISR — Incremental Static Regeneration

- How it works:
 1. Like SSG, pages are pre-rendered at build.
 2. When a user requests a page after revalidate seconds:
 - Server serves the cached static page immediately.
 - In the background, Next.js regenerates the page and updates the cache.
- Key point: Static pages + auto-update in background — best of SSG + SSR.

Pros:

- Fast + updatable content without full rebuild.
- Best of SSG + SSR.

Cons:

- Slight caching complexity.
- Might serve stale content until regeneration completes.

Hydration

- What it is: The process where static HTML sent by the server (SSR/SSG) becomes a fully interactive React app in the browser.
 - Essentially, the browser “wakes up” the HTML by attaching event listeners and JS behavior.
-

How it works:

- 1. Server sends ready-to-display HTML (SSR/SSG).**
- 2. Browser renders the HTML immediately.**
- 3. React JS runs in the background, connecting the HTML with React components.**
- 4. Page becomes fully interactive (buttons, forms, etc.).**

Why it matters:

- Without hydration → page looks correct but cannot respond to clicks or JS events.**
- Hydration bridges the fast first render of SSR/SSG with React's dynamic interactivity.**

Create a Next.js Project

`npx create-next-app@latest my-next-app`

It will ask a few questions:

- TypeScript or JavaScript → choose based on preference.
- ESLint, Tailwind, etc. → optional.

Navigate to your project:

`cd my-next-app`

`npm run dev`

- Opens the dev server at `http://localhost:3000` → your first live Next.js app.

Folder Structure

`my-next-app/`

|

└─ node_modules/ # Installed packages

└─ public/ # Static files (images, fonts)

└─ src/

| └─ app/ # Main app folder (Next 13+ App Router)

| | └─ page.tsx # Default home page

| | └─ layout.tsx # Common layout for all pages

| └─ styles/ # CSS/SCSS files

└─ package.json # Project config & dependencies

└─ next.config.js # Next.js configuration

└─ tsconfig.json # TypeScript config (if TS project)

└─ .eslintrc.json # Linting rules

src

```
└─ app
|   └─ page.tsx    # home page
|   └─ about
|       └─ page.tsx / about page
|   └─ api
|       └─ users
|           └─ route.ts /route endpoint
|   └─ layout.tsx
|
└─ components
    └─ Navbar.tsx
|
└─ styles
```

Components in Next.js

Next.js components are React components, but they can run in two environments:

1. Server Components (default)
2. Client Components

1 Server Components (Default)

If you create a component in Next.js without "use client", it becomes a Server Component.

How they behave:

- Run on the server

- Send HTML to the browser
- Reduce JavaScript sent to the client
- Can directly fetch data (DB, API)

Limitations:

- ❌ No useState
- ❌ No useEffect
- ❌ No event handlers (onClick, etc.)

2 Client Components

If you need interactivity, you must mark the component with:

"use client";

This tells Next.js this component should run in the browser.

Capabilities:

- ✅ useState
- ✅ useEffect
- ✅ Event handlers
- ✅ Browser APIs

Why Next.js Uses This System

Server components:

- reduce bundle size
- improve performance
- allow direct data fetching

Client components:

- enable UI interactivity

So Next.js splits work between server and browser.

Page ,Layout and Template

1 page.tsx

What it is:

Represents the actual UI of a route.

Example:

`app/dashboard/page.tsx`

Route:

`/dashboard`

Example code:

```
export default function Page() {  
  return <h1>Dashboard Page</h1>;  
}
```

Key idea:

- Each route must have a page.tsx
- It defines what the user sees

2 layout.tsx

What it is:

A shared wrapper UI for pages.

Used for things like:

- Navbar
- Sidebar
- Footer

Layouts persist between page navigations.

Example structure:

```
app/  
└─ layout.tsx
```



```
└─ page.tsx
└─ dashboard/
    └─ layout.tsx
        └─ page.tsx
```

Example layout:

```
export default function Layout({ children }) {
  return (
    <div>
      <nav>Navbar</nav>
      {children}
    </div>
  );
}
```

Here:

- Navbar stays
- Only page content changes

Key idea:

Layout does NOT re-render during navigation.

In Next.js layouts we don't manually pass children. Next.js automatically injects the current page or nested route as children inside the layout."

3 template.tsx

template.tsx is similar to layout, but with one major difference.

It re-renders on every navigation.

Example:

app/dashboard/template.tsx

Example:

```
export default function Template({ children }) {
  return (
```

```
<div>
  <h2>Dashboard Wrapper</h2>
  {children}
</div>
);
}
```

Use when you want things like:

- animations
- resetting state
- fresh render every route change

“page.tsx defines the UI for a route. layout.tsx is a shared wrapper that persists across navigation, commonly used for things like navbars and sidebars. template.tsx is similar to layout but re-renders on every navigation, which is useful for animations or resetting component state.

File Based Routing

In Next.js, your folder structure automatically becomes your URL structure.

You don't write route configs like React Router.

Folders = routes

page.tsx = UI for that route

Project structure:

```
app/
├─ page.tsx
├─ about/
│   └─ page.tsx
└─ contact/
    └─ page.tsx
```

Routes created automatically:

File	Route
app/page.tsx	/
app/about/page.tsx	/about
app/contact/page.tsx	/contact

3 Nested Routes

You can create nested URLs with folders.

Structure:

```
app/  
  └─ blog/  
    └─ post/  
      └─ page.tsx
```

Route becomes:

/blog/post

4 Dynamic Routes

If a route needs a variable value (id, slug) you use square brackets.

Example:

```
app/  
  └─ blog/  
    └─ [id]/  
      └─ page.tsx
```

Routes:

/blog/1
/blog/42
/blog/nextjs-routing

Example code:

```
export default function Page({ params }) {  
  return <h1>Blog ID: {params.id}</h1>;  
}
```

6 API Routes

APIs also follow file-based routing.

Structure:

```
app/  
└─ api/  
    └─ users/  
        └─ route.ts
```

Route:

/api/users

Example:

```
export async function GET() {  
  return Response.json({ message: "Hello API" });  
}
```

File-based routing in Next.js means routes are automatically generated from the folder and file structure inside the app directory, where page.tsx defines the UI for that route."

Navigation

Next.js uses the Link component for navigation instead of <a> tags.

It enables client-side navigation, meaning no full page reload.

Example:

```
import Link from "next/link";  
  
export default function Page() {  
  return (  

```

```
<div>
  <Link href="/about">Go to About</Link>
</div>
);
}
```

Route:

/about

Why Link instead of <a>?

<a> causes full page reload.

Link enables client-side navigation, which means:

- Faster navigation
- No full reload
- Better user experience

3 Programmatic Navigation (useRouter)

Sometimes navigation happens after an action (form submit, login, etc.).

For that we use router navigation.

Example:

```
"use client";
```

```
import { useRouter } from "next/navigation";
```

```
export default function Page() {
  const router = useRouter();
```

```
  function handleClick() {
    router.push("/dashboard");
  }
```

```
  return <button onClick={handleClick}>Go Dashboard</button>;
}
```

1 loading.tsx

What it is

loading.tsx shows a loading UI while a page or data is loading.

Next.js automatically displays it when:

- a route is loading
- server components are fetching data
- navigation is happening

Example Structure

```
app/  
└─ dashboard/  
    └─ page.tsx  
        └─ loading.tsx
```

loading.tsx

```
export default function Loading() {  
  return <h2>Loading...</h2>;  
}
```

What happens

User visits:

/dashboard

Flow:

1. loading.tsx appears
2. page.tsx finishes loading
3. real page replaces loading UI

Mental Model

loading.tsx = automatic loading spinner for that route

You don't manually control it.

2 error.tsx

What it is

error.tsx shows a fallback UI when an error happens in a route.

It works like a React error boundary.

Example Structure

```
app/  
├─ dashboard/  
│   └─ page.tsx  
│       └─ error.tsx
```

error.tsx

"use client";

```
export default function Error({ error, reset }) {  
  return (  
    <div>  
      <h2>Something went wrong</h2>  
      <button onClick={() => reset()}>  
        Try again  
      </button>  
    </div>  
  );  
}
```

What happens

If page.tsx crashes:

Error occurs → error.tsx UI appears

User can click reset() to retry.

Important Detail

error.tsx must be a client component.

That's why it needs:

"use client"

Because it uses:

- **event handlers**
- **state**

“loading.tsx is used to display a loading UI while a route or data is being fetched, and error.tsx acts as an error boundary that renders fallback UI when an error occurs in that route segment.

Q: Why are Server Components useful?

Strong Answer

Server Components reduce the amount of JavaScript sent to the browser, improve performance, and allow direct access to backend resources like databases without exposing them to the client.

Q.What is Build Time?

Answer

Build time is when the application is compiled and optimized for production, typically when running `npm run build`. Static pages are generated during this phase.

Q. What does "use client" do?

Answer

"use client" tells Next.js that the component should run in the browser instead of the server, enabling features like `useState`, `useEffect`, and event handlers.

Q.What are Server Components?

Answer

Server Components run on the server and send HTML to the client. They help reduce JavaScript bundle size and can directly fetch data from APIs or databases.

Limitations:

- No hooks
 - No event handlers
-

Q. What are Client Components?

Answer

Client Components run in the browser and allow interactivity like state, hooks, and event handlers. They must be marked using "use client".